

Лабораторная работа №7

Введение в работу с данными

Клюкин М. А.

Российский университет дружбы народов, Москва, Россия

- Ключин Михаил Александрович
- студент
- Российский университет дружбы народов
- 1132226431@pruf.ru
- <https://MaKYaro.github.io/ru/>



Цели и задачи

Основной целью работы является специализированных пакетов Julia для обработки данных(**julia~~lang~~?**).

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы(**juliadoc?**).

Выполнение лабораторной работы

В Julia для обработки данных используются наработки из других языков программирования, в частности, из R и Python.

```
# Обновление окружения:  
using Pkg  
Pkg.update  
# Установка пакетов:  
using Pkg  
for p in ["CSV", "DataFrames", "RDatasets", "FileIO"]  
Pkg.add(p)  
end  
using CSV, DataFrames, DelimitedFiles
```

Рис. 1: Установка пакетов

Считывание данных

```
# Считывание данных и их запись в структуру:
P = CSV.File("programminglanguages.csv") |> DataFrame
```

Row	year	language
	Int64	String31
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN
6	1957	COMTRAN
7	1958	LISP
8	1958	ALGOL 58
9	1959	FACT

```
# Функция определения по названию языка программирования года его создания:
function language_created_year(P, language::String)
    loc = findfirst(P[:,2].==language)
    return P[loc,1]
end

language_created_year (generic function with 1 method)

# Пример вызова функции и определение даты создания языка Python:
language_created_year(P,"Python")

1991
```

Рис. 2: Считывание данных и запись в структуру

```
# Пример вызова функции и определение даты создания языка Julia:  
language_created_year(P, "Julia")
```

2012

Рис. 3: Пример

```
language_created_year(P,"julia")

MethodError: no method matching getindex(::DataFrame, ::Nothing, ::Int64)
The function `getindex` exists, but no method is defined for this combination of argument
types.

Closest candidates are:
  getindex(::DataFrame, ::Colon, ::Union{AbstractString, Signed, Symbol, Unsigned})
    @ DataFrames ~/.julia/packages/DataFrames/b4w9K/src/dataframe/dataframe.jl:542
  getindex(::DataFrame, ::typeof(!), ::Union{Signed, Unsigned})
    @ DataFrames ~/.julia/packages/DataFrames/b4w9K/src/dataframe/dataframe.jl:548
  getindex(::DataFrame, ::InvertedIndex, ::Union{AbstractString, Signed, Symbol, Unsigned})
    @ DataFrames ~/.julia/packages/DataFrames/b4w9K/src/dataframe/dataframe.jl:538
  ...

Stacktrace:
 [1] language_created_year(P::DataFrame, language::String)
    @ Main ./In[3]:4
 [2] top-level scope
    @ In[6]:1
 [3] eval(m::Module, e::Any)
    @ Core ./boot.jl:489
```

Рис. 4: Поиск “julia” со строчной буквы

Считывание данных

```
# Функция определения по названию языка программирования  
# года его создания (без учёта регистра):  
function language_created_year_v2(P, language::String)  
    loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))  
    return P[loc,1]  
end
```

```
language_created_year_v2 (generic function with 1 method)
```

```
# Пример вызова функции и определение даты создания языка julia:  
language_created_year_v2(P, "julia")
```

```
2012
```

Рис. 5: Изменение исходной функции

```
# Построчное считывание данных с указанием разделителя:  
Tx = readln("programmingLanguages.csv", ',')
```

```
74x2 Matrix[Any]:  
  "year"  "Language"  
1951      "Regional Assembly Language"  
1952      "Autocode"  
1954      "IPL"  
1955      "FLOW-MATIC"  
1957      "FORTRAN"  
1957      "COMTRAN"  
1958      "LISP"  
1958      "ALGOL 58"  
1959      "FACT"  
1959      "COBOL"  
1959      "RPG"  
1962      "APL"  
:  
2003      "Scala"  
2005      "F#"  
2006      "PowerShell"  
2007      "Clojure"  
2009      "Go"  
2010      "Rust"  
2011      "Dart"  
2011      "Kotlin"  
2011      "Red"  
2011      "Elixir"  
2012      "Julia"  
2014      "Swift"
```

Рис. 6: Построчное считывание данных

```
# Запись данных в CSV-файл:  
CSV.write("programming_languages_data2.csv", P)  
  
"programming_languages_data2.csv"
```

Рис. 7: Запись данных в файл

```
# Пример записи данных в текстовый файл с разделителем ',':  
writedlm("programming_languages_data.txt", Tx, ',')  
# Пример записи данных в текстовый файл с разделителем '-':  
writedlm("programming_languages_data2.txt", Tx, '-')
```

Рис. 8: Пример с указанием типа данных и разделителем данных

Запись данных в файл

```
# Построчное считывание данных с указанием разделителя:  
P_new_delim = readLines("programming_languages_data2.txt", "-")  
  
74x2 Matrix(Any):  
  "year"  "language"  
1951      "Regional Assembly Language"  
1952      "Autocode"  
1954      "IPL"  
1955      "FLOW-MATIC"  
1957      "FORTRAN"  
1957      "COMTRAN"  
1958      "LISP"  
1958      "ALGOL 58"  
1959      "FACT"  
1959      "COBOL"  
1959      "RPG"  
1962      "APL"  
:  
2003      "Scala"  
2005      "F#"  
2006      "PowerShell"  
2007      "Clojure"  
2009      "Go"  
2010      "Rust"  
2011      "Dart"  
2011      "Kotlin"  
2011      "Red"  
2011      "Elixir"  
2012      "Julia"  
2014      "Swift"
```

Рис. 9: Проверка корректности считывания созданного текстового файла


```
# Инициализация словаря:  
dict = Dict{Integer,Vector{String}}()  
  
Dict{Integer, Vector{String}}()
```

Рис. 10: Инициализация словаря

```
# Инициализация словаря:
```

```
dict2 = Dict()
```

```
Dict{Any, Any}()
```

Рис. 11: Инициализация пустого словаря

```
# Заполнение словаря данными:  
for i = 1:size(P,1)  
    year,lang = P[i,:]  
    if year in keys(dict)  
        dict[year] = push!(dict[year],lang)  
    else  
        dict[year] = [lang]  
    end  
end
```

Рис. 12: Заполнение словаря данными

```
# Пример определения в словаре языков программирования, созданных в 2003 году  
dict[2003]
```

```
2-element Vector{String}:  
 "Groovy"  
 "Scala"
```

Рис. 13: Пример работы словаря

```
# Подгружаем пакет DataFrames:
using DataFrames
# Задаём переменную со структурой DataFrame:
df = DataFrame(year = P[:,1], language = P[:,2])
```

73×2 DataFrame 48 r

Row	year	language
	Int64	String31
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN
6	1957	COMTRAN
7	1958	LISP
8	1958	ALGOL 58

Рис. 14: Пример создания структуры DataFrame

```
# Вывод всех значений столбца year:
df[1,year]

73-element Vector{Int64}:
 1951
 1952
 1954
 1955
 1957
 1957
 1957
 1958
 1958
 1959
 1959
 1962
 1962
 1962
 1963
 2003
 2005
 2006
 2007
 2009
 2010
 2011
 2011
 2011
 2011
 2012
 2014

# Получение статистических сведений о фрейме:
describe(df)

2x7 DataFrame
  Row variable  mean   min    median  max  nmissing  eltype
  Symbol Union... Any    Union... Any    Int64    DataType
1 year      1982.99 1951    1960.0  2014      0      Int64
```

Рис. 15: Пример создания структуры DataFrame

```
# Подгружаем пакет Rdatasets:  
using Rdatasets  
# Задаем структуру данных в виде набора данных:  
iris = dataset("datasets", "iris")
```

150x5 DataFrame 125 rows

Row	SepalLength Float64	SepalWidth Float64	PetalLength Float64	PetalWidth Float64	Species Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3.0	1.4	0.1	setosa
139	6.0	3.0	4.8	1.8	virginica
140	6.9	3.1	5.4	2.1	virginica
141	6.7	3.1	5.6	2.4	virginica
142	6.9	3.1	5.1	2.3	virginica

Рис. 16: Работа с пакетом RDatasets

```
# Определения типа переменной:
typeof(iris)

DataFrame

describe(iris)

5x7 DataFrame
```

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	SepalLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepalWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetalLength	3.758	1.0	4.35	6.9	0	Float64
4	PetalWidth	1.19933	0.1	1.3	2.5	0	Float64
5	Species		setosa		virginica	0	CategoricalValue{String, UInt8}

Рис. 17: Получение основных статических сведений о каждом столбце в наборе данных

Работа с переменными отсутствующего типа (Missing Values)

```
# Отсутствующий тип:
```

```
a = missing
```

```
typeof(a)
```

```
Missing
```

```
# Пример операции с переменной отсутствующего типа:
```

```
a + 1
```

```
missing
```

Рис. 18: Использование “отсутствующего” типа. Операция сложения числа и переменной с отсутствующим типом

Работа с переменными отсутствующего типа (Missing Values)

```
: # Определение перечня продуктов:
foods = ["apple", "cucumber", "tomato", "banana"]
# Определение калорий:
calories = [missing, 47, 22, 105]

: 4-element Vector{Union{Missing, Int64}}:
   missing
    47
    22
   105

: # Определение типа переменной:
typeof(calories)

: Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})
```

Рис. 19: Пример работы с данными, среди которых есть данные с отсутствующим типом

Работа с переменными отсутствующего типа (Missing Values)

```
# Подключаем пакет Statistics:  
using Statistics  
# Определение среднего значения:  
mean(calories)
```

missing

```
# Определение среднего значения без значений с отсутствующим типом:  
mean(skipmissing(calories))
```

58.0

Рис. 20: Игнорирование отсутствующего типа

Работа с переменными отсутствующего типа (Missing Values)

```
# Задание сведений о ценах:
prices = [0.85, 1.6, 0.8, 0.6]
# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods, calories=calories)
# Формирование данных о ценах:
dataframe_prices = DataFrame(item=foods, price=prices)
# Объединение данных о калориях и ценах:
DF = innerjoin(dataframe_calories, dataframe_prices, on=:item)
```

4×3 DataFrame

Row	item	calories	price
	String	Int64?	Float64
1	apple	missing	0.85
2	cucumber	47	1.6
3	tomato	22	0.8
4	banana	105	0.6

Рис. 21: Формирование таблиц данных и их объединение в один фрейм

```
# Подключаем пакет FileIO:
```

```
using FileIO
```

```
# Подключаем пакет ImageIO:
```

```
import Pkg
```

```
Pkg.add("ImageIO")
```

```
Resolving package versions...
```

```
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`
```

```
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`
```

```
# Загрузка изображения:
```

```
X1 = load("julia.png")
```

Рис. 22: Подключение пакетов. Загрузка изображения

Обработка данных: стандартные алгоритмы машинного обучения в Julia.

Кластеризация данных. Метод k-средних

```
# Загрузка данных:  
houses = CSV.File("houses.csv") |> DataFrame
```

985×12 DataFrame *960 rows omitted*

Row	street	city	zip	state	beds	baths	sq_ft	type	sale_date	price
	String	String15	Int64	String3	Int64	Int64	Int64	String15	String31	Int64
1	3526 HIGH ST	SACRAMENTO	95838	CA	2	1	836	Residential	Wed May 21 00:00:00 EDT 2008	5922
2	51 OMAHA CT	SACRAMENTO	95823	CA	3	1	1167	Residential	Wed May 21 00:00:00 EDT 2008	6821
3	2796 BRANCH ST	SACRAMENTO	95815	CA	2	1	796	Residential	Wed May 21 00:00:00 EDT 2008	6888

Рис. 23: Загрузка данных

Обработка данных: стандартные алгоритмы машинного обучения в Julia.

Кластеризация данных. Метод k-средних

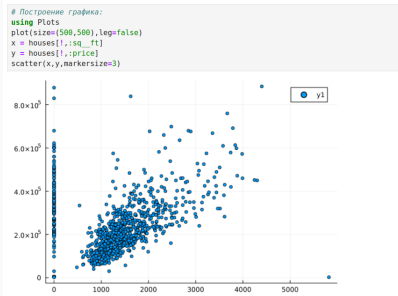


Рис. 24: Построение графика цен на недвижимость в зависимости от площади

Обработка данных: стандартные алгоритмы машинного обучения в Julia.

Кластеризация данных. Метод k-средних

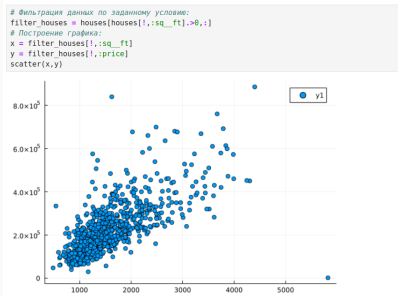


Рис. 25: Построение графика без “артефактов”

Обработка данных: стандартные алгоритмы машинного обучения в Julia.

Кластеризация данных. Метод k-средних

```
# Подключение пакета Statistics:  
using Statistics  
# Определение средней цены для определённого типа домов:  
combine(groupby(filter_houses, :type), :price => mean)
```

3×2 DataFrame

Row	type	price_mean
	String15	Float64
1	Residential	2.34802e5
2	Condo	1.34213e5
3	Multi-Family	2.24535e5

Рис. 26: Среднее значение

Обработка данных: стандартные алгоритмы машинного обучения в Julia.

Кластеризация данных. Метод k-средних

```
import Pkg
Pkg.add("Clustering")
using Clustering
# Добавление данных :latitude и :longitude в новый фрейм:
X = filter_houses[:, [:latitude, :longitude]]

# Преобразование DataFrame в матрицу
X = Matrix{Float64}(X)
k = length(unique(filter_houses[:, :zip]))

# Определение кластеров k-среднего
C = kmeans(X, k)

# Формирование DataFrame с результатами кластеризации
df = DataFrame{
    cluster = C.assignments,
    city = filter_houses[:, :city],
    latitude = filter_houses[:, :latitude],
    longitude = filter_houses[:, :longitude],
    zip = filter_houses[:, :zip]
}

# Построение графика кластеров
clusters_figure = plot(legend=false)
for i in 1:k
    clustered_houses = df[df[:, :cluster] .== i, :]
    xvals = clustered_houses[:, :latitude]
    yvals = clustered_houses[:, :longitude]
    scatter!(clusters_figure, xvals, yvals, markersize=4)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by cluster")
display(clusters_figure)
```

Рис. 27: Построение графика с кластерами разных цветов

Обработка данных: стандартные алгоритмы машинного обучения в Julia.

Кластеризация данных. Метод k-средних

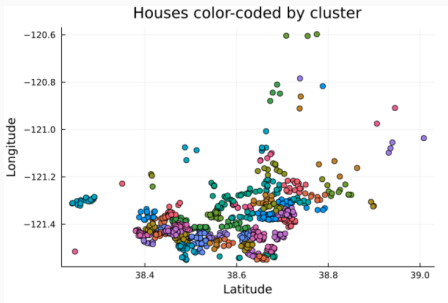


Рис. 28: Построение графика с кластерами разных цветов

Обработка данных: стандартные алгоритмы машинного обучения в Julia.

Кластеризация данных. Метод k-средних

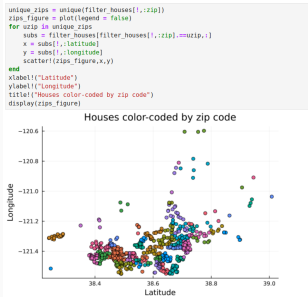


Рис. 29: Построение графика с кластерами разных цветов по почтовому индексу

Кластеризация данных. Метод k ближайших соседей

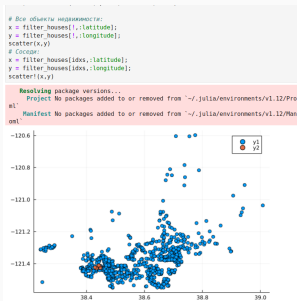


Рис. 30: Отображение на графике соседей выбранного объекта недвижимости

Кластеризация данных. Метод k ближайших соседей

```
# Фильтрация по районам соседних домов:  
cities = filter_houses[idxs,:city]
```



```
10-element PooledArrays.PooledVector{String15, UInt32, Vector{UInt32}}:  
"SACRAMENTO"  
"ELK GROVE"  
"SACRAMENTO"  
"SACRAMENTO"  
"SACRAMENTO"  
"SACRAMENTO"  
"ELK GROVE"  
"ELK GROVE"  
"ELK GROVE"  
"ELK GROVE"
```

Рис. 31: Определение районов соседних домов

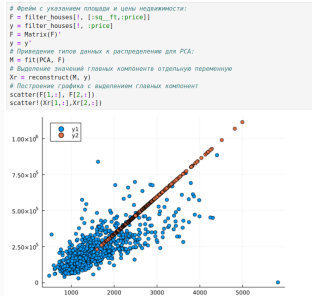


Рис. 32: Попытка уменьшения размера данных о цене и площади из набора данных ДОМОВ

Обработка данных. Линейная регрессия

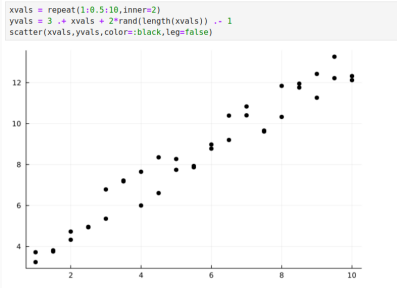


Рис. 33: Исходные данные

Обработка данных. Линейная регрессия

```
function find_best_fit(xvals,yvals)
    meanx = mean(xvals)
    meany = mean(yvals)
    stdx = std(xvals)
    stdy = std(yvals)
    r = cor(xvals,yvals)
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
end
```

find_best_fit (generic function with 1 method)

```
a,b = find_best_fit(xvals,yvals)
ynew = a * xvals .+ b
plot(xvals,ynew)
```

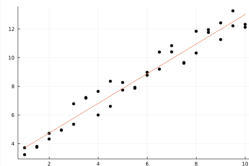


Рис. 34: Применение функции для построения графика

Обработка данных. Линейная регрессия

```
: xvals = 1:100000;  
xvals = repeat(xvals,inner=3);  
yvals = 3 .* xvals + 2*rand(length(xvals)) .* 1;  
@show size(xvals)  
@show size(yvals)  
  
size(xvals) = (300000,)  
size(yvals) = (300000,) : (300000,)  
  
: @time a,b = find_best_fit(xvals,yvals)  
  
0.020382 seconds (4.38 k allocations: 230.047 KiB, 85.43% compilation time)  
: (1.0000000396709656, 3.000216230582737)  
  
: import Pkg  
Pkg.add("PyCall")  
Pkg.add("Conda")  
using PyCall  
using Conda
```

Рис. 35: Генерация данных. Определение времени соответствия

Обработка данных. Линейная регрессия

```
py"""
import numpy
def find_best_fit_python(xvals,yvals):
    meanx = numpy.mean(xvals)
    meany = numpy.mean(yvals)
    stdx = numpy.std(xvals)
    stdy = numpy.std(yvals)
    r = numpy.corrcoef(xvals,yvals)[0][1]
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
"""

xpy = PyObject(xvals)
ypy = PyObject(yvals)
@time a,b = py.find_best_fit_python(xpy,ypy)

Resolving package versions...
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`.
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`.
Resolving package versions...
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`.
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`.
0.000980 seconds (160 allocations: 3.641 KiB)
[1.0000000396709676, 3.0002162394808735]

import Pkg
Pkg.add("BenchmarkTools")
using BenchmarkTools
@time a,b = py.find_best_fit_python(xvals,yvals)
@time a,b = find_best_fit(xvals,yvals)

Resolving package versions...
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`.
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`.
4.736 ms (34 allocations: 960 bytes)
1.012 ms (1 allocation: 32 bytes)
[1.0000000396709658, 3.000216239582737]
```

Рис. 36: Сравнение

```
using RDatasets, Clustering, Plots
iris = dataset("datasets", "iris")

iris = dataset("datasets", "iris")
iris_spec = Dict{"setosa"=>1, "versicolor"=>2, "virginica"=>3}
iris[!, :5] = [iris_spec[item] for item in iris[!, :5]]

X = Matrix(iris)'
k = length(unique(iris[!, :5]))
C = kmeans(X, k)
df = DataFrame(cluster = C.assignments, SepalLength = iris[!, :SepalLength],
               SepalWidth = iris[!, :SepalWidth], PetalLength = iris[!, :PetalLength],
               PetalWidth = iris[!, :PetalWidth], Species = iris[!, :Species]
               )

clusters_figure = plot(legend = false)
for i = 1:k
    clustered_houses = df[df[!, :cluster].== i, :]
    xvals = clustered_houses[!, :SepalWidth]
    yvals = clustered_houses[!, :SepalLength]
    scatter!(clusters_figure, xvals, yvals, markersize = 4)
end
xlabel!("SepalWidth")
ylabel!("SepalLength")
title!("Iris. Cluster")
display(clusters_figure)
```

Рис. 37: Кластеризация

Самостоятельное выполнение

```
clusters_figure = plot(legend = false)
for i = 1:k
    clustered_houses = df[df[:, :cluster].== i, :]
    xvals = clustered_houses[:, :SepalWidth]
    yvals = clustered_houses[:, :SepalLength]
    scatter!(clusters_figure, xvals, yvals, markersize = 4)
end
xlabel!("SepalWidth")
ylabel!("SepalLength")
title!("Iris. Cluster")
display(clusters_figure)
```

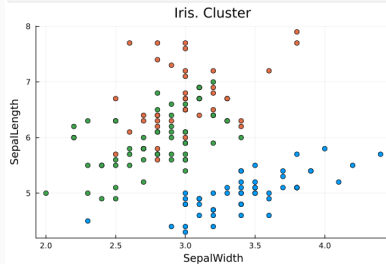


Рис. 38: Кластеризация

Самостоятельное выполнение

```
clusters_figure = plot(legend=false)
for i = 1:k
    clustered_houses = df[df[:, :cluster].== i, :]
    xvals = clustered_houses[:, :PetalWidth]
    yvals = clustered_houses[:, :PetalLength]
    scatter!(clusters_figure, xvals, yvals, markersize = 4)
end
xlabel!("PetalWidth")
ylabel!("PetalLength")
title!("Iris. Cluster")
display(clusters_figure)
```

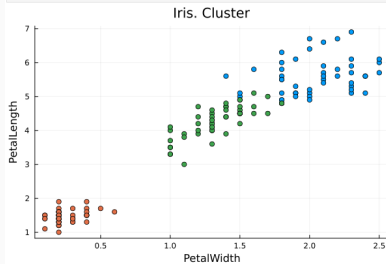


Рис. 39: Кластеризация

```
X = randn(1000, 3)
a0 = rand(3)
y = X * a0 + 0.1 * randn(1000)
scatter(X, y, vals = (1, 2, 3))
```

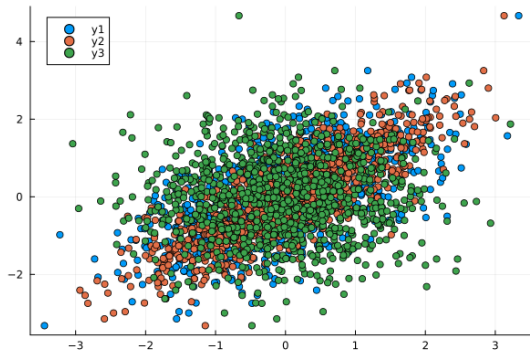


Рис. 40: Регрессия

Самостоятельное выполнение

```
X2 = hcat(ones(1000), X)
beta = (X2' * X2) \ (X2' * y)
```

```
4-element Vector{Float64}:
-0.00286621504429712
 0.6341176413533149
 0.8532612112348272
 0.03153307554101222
```

```
using MultivariateStats
beta_mvs = llsq(X, y)
```

```
4-element Vector{Float64}:
 0.6341176413533149
 0.8532612112348268
 0.031533075541012215
-0.0028662150442970947
```

```
Pkg.add("GLM")
using GLM
df = DataFrame(hcat(X, y), :auto)
rename!(df, ["x1", "x2", "x3", "y"])
model = lm(@formula(y ~ x1 + x2 + x3), df)
coef(model)
```

```
Resolving package versions...
```

```
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`
```

```
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`
```

```
4-element Vector{Float64}:
-0.0028662150442971034
 0.6341176413533149
 0.853261211234827
 0.031533075541012215
```

Рис. 41: Регрессия


```
X = rand(100)
y = 2X + 0.1 * randn(100)
a, b = find_best_fit(X, y)
scatter(X, y, c = "pink", title = "График регрессии", xlabel='x', ylabel='y', leg=false)
Plots.abline!(a, b, lw=2)
```

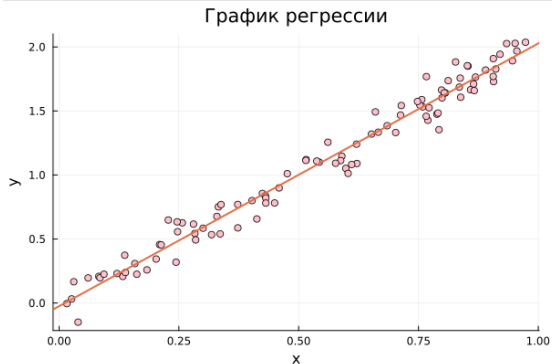


Рис. 42: Регрессия

Построим траекторию возможных цен на акции:

- S – начальная цена акции;
- T – длина биномиального дерева в годах;
- n – количество периодов;
- $h = Tn$ – длина одного периода;
- σ – волатильность акции;
- r – годовая процентная ставка;
- $u = \exp(rh + \sigma\sqrt{h})$;
- $d = \exp(rh - \sigma\sqrt{h})$;
- $p^* = \frac{\exp(rh) - d}{u - d}$;

Самостоятельное выполнение

```
: S = 100.0
  T = 1.0
  n = 1000
  sigma = 0.3
  r = 0.08
  h = T / n

  u = exp(r * h + sigma * sqrt(h))
  d = exp(r * h - sigma * sqrt(h))
  p = (exp(r * h) - d) / (u - d)

function create_path(S, r, sigma, T, n)
    path = []
    S_ = S
    push!(path, S)
    for i in 1:n
        if rand() < p
            S_ *= u
        else
            S_ *= d
        end
        push!(path, S_)
    end
    return path
end

path = create_path(S, r, sigma, T, n)
plot(path, label="Stock Price Path", xlabel="Time Step", ylabel="Price", title="Stock Price Trajectory")
```

Рис. 43: Модель ценообразования биномиальных опционов

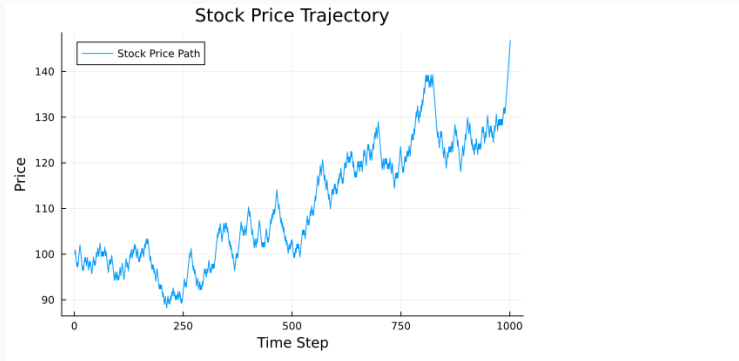


Рис. 44: Модель ценообразования биномиальных опционов

```
function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    Y = Vector{Float64}()
    X = collect(0:T/n:1)
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p = (exp(r * h) - d) / (u - d)
    for i = 0:n
        prob = rand()
        if prob < p
            S *= u
        else
            S *= d
        end
        push!(Y, S)
    end
    return(X, Y)
end

p = plot()
for i = 1:10
    plot!(p, createPath(S, r, sigma, T, n))
end
p
```

Рис. 45: Модель ценообразования биномиальных опционов

Самостоятельное выполнение

```
p = plot()
for i = 1:10
    plot!(p, createPath(S, r, sigma, T, n))
end
p
```

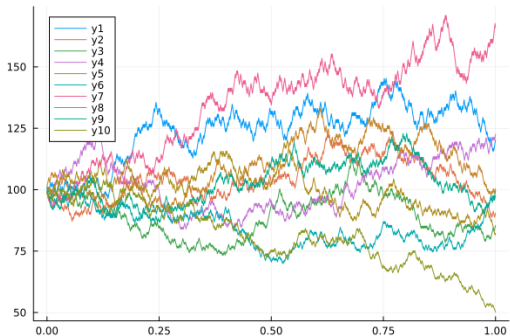


Рис. 46: Модель ценообразования биномиальных опционов

```
function createPath_(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    Y = Vector{Float64}()
    X = collect(0:T/n:1)
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p = (exp(r * h) - d) / (u - d)
    Threads.@threads for i = 0:n
        prob = rand()
        if prob < p
            S *= u
        else
            S *= d
        end
        push!(Y, S)
    end
    return(X, Y)
end

p = plot()
for i = 1:10
    plot!(p, createPath_(S, r, sigma, T, n))
end
p
```

Рис. 47: Модель ценообразования биномиальных опционов

```
p = plot()
for i = 1:10
    plot!(p, createPath(S, r, sigma, T, n))
end
p
```

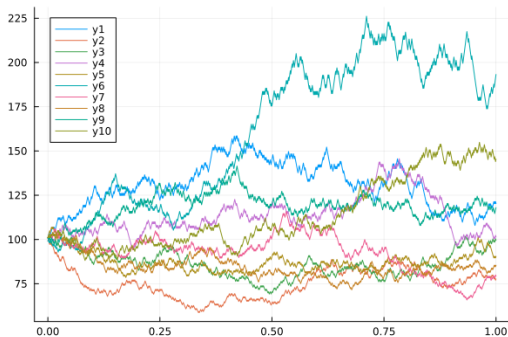


Рис. 48: Модель ценообразования биномиальных опционов

В ходе выполнения лабораторной работы были изучены специализированные пакеты Julia для обработки данных.